

MileStone 1 – SQL Sample Questions

FULL PRACTICE PDF: database/table creation queries + INSERT queries + problem requirements + answer queries + expected outputs.

Source basis: the uploaded 22-page MileStone 1 Sample Questions SQL PDF. The source has 6 questions: 3 Easy and 3 Medium.

■filecite■turn4file0■L13-L22■

Q1. EASY – Customer with the Highest Value Order

Requirement: Find the customer who placed the single highest-value order. Return order_id, customer_name and total_amount. The original question explicitly asks for a single-row subquery.

A. Database + ALL TABLE CREATION QUERIES

```
CREATE DATABASE retail_store;
USE retail_store;

CREATE TABLE Customers (
  customer_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  phone VARCHAR(20)
);

CREATE TABLE Products (
  product_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  category VARCHAR(50) NOT NULL,
  price DECIMAL(10,2) NOT NULL
);

CREATE TABLE Orders (
  order_id INT PRIMARY KEY,
  customer_id INT,
  order_date DATE NOT NULL,
  total_amount DECIMAL(10,2),
  FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);

CREATE TABLE Order_Items (
  order_id INT,
  product_id INT,
  quantity INT NOT NULL,
  unit_price DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (order_id, product_id),
  FOREIGN KEY (order_id) REFERENCES Orders(order_id),
  FOREIGN KEY (product_id) REFERENCES Products(product_id),
  FOREIGN KEY (order_id) REFERENCES Orders(order_id)
);
```

B. ALL INSERT QUERIES – source test data

```
INSERT INTO Customers VALUES
(201, 'John Doe', 'john@example.com', '1234567890'),
(202, 'Jane Smith', 'jane@example.com', '2345678901'),
(203, 'Emily Davis', 'emily@example.com', '3456789012'),
(204, 'Mark Wilson', 'mark@example.com', '4567890123');

INSERT INTO Products VALUES
(301, 'Laptop', 'Electronics', 3000.00),
(302, 'Smartphone', 'Electronics', 600.00),
(303, 'Office Chair', 'Furniture', 150.00),
(304, 'Notebook Set', 'Stationery', 30.00);

INSERT INTO Orders VALUES
(401, 201, '2023-08-01', 3010.00),
(402, 202, '2023-08-03', 1600.00),
(403, 203, '2023-08-04', 20.00),
(404, 204, '2023-08-05', 750.00);

INSERT INTO Order_Items VALUES
(401, 301, 1, 3000.00),
(401, 304, 1, 10.00),
(402, 302, 1, 600.00),
(402, 301, 1, 1000.00),
(403, 304, 2, 10.00),
(404, 303, 1, 150.00),
(404, 302, 1, 600.00);
```

C. ANSWER QUERY

```
SELECT
  o.order_id,
  c.name AS customer_name,
  o.total_amount
```

```
FROM Orders o
JOIN Customers c
  ON o.customer_id = c.customer_id
WHERE o.total_amount = (
  SELECT MAX(total_amount)
  FROM Orders
);
```

order_id	customer_name	total_amount
401	John Doe	3010.00

Q2. EASY – Student Enrollments with Grades and Department Names

Requirement: Return student_id, student_name, course_id, course_name, grade, and department_name (the department offering the course).

A. DATABASE + ALL TABLE CREATION QUERIES

```
CREATE DATABASE university_core;
USE university_core;

CREATE TABLE Departments (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(50) NOT NULL
);

CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    age INT,
    department_id INT,
    FOREIGN KEY (department_id) REFERENCES Departments(department_id)
);

CREATE TABLE Courses (
    course_id VARCHAR(10) PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    department_id INT,
    FOREIGN KEY (department_id) REFERENCES Departments(department_id)
);

CREATE TABLE Enrollments (
    student_id INT,
    course_id VARCHAR(10),
    grade VARCHAR(5),
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

B. ALL INSERT QUERIES – source test data

```
INSERT INTO Departments VALUES
(1, 'Computer Science'),
(2, 'Mathematics'),
(3, 'Physics'),
(4, 'Literature');

INSERT INTO Students VALUES
(101, 'Alice Smith', 20, 1),
(102, 'Bob Johnson', 22, 2),
(103, 'Clara Evans', 21, 3),
(104, 'David Kumar', 23, 1);

INSERT INTO Courses VALUES
('C101', 'Algorithms', 1),
('C102', 'Linear Algebra', 2),
('C103', 'Quantum Mechanics', 3),
('C104', 'Shakespearean Drama', 4);

INSERT INTO Enrollments VALUES
(101, 'C101', 'A'),
(101, 'C102', 'B+'),
(101, 'C104', 'A-'),
(102, 'C102', 'B'),
(102, 'C103', 'A'),
(103, 'C103', 'B-'),
(103, 'C104', 'A'),
(104, 'C101', 'A+'),
(104, 'C103', 'B'),
(104, 'C102', 'C+'),
(104, 'C104', 'B'),
(101, 'C103', 'B+');
```

C. ANSWER QUERY

```
SELECT
    s.student_id,
    s.name AS student_name,
    c.course_id,
    c.course_name,
    e.grade,
    d.department_name
FROM Students s
JOIN Enrollments e
    ON s.student_id = e.student_id
JOIN Courses c
    ON e.course_id = c.course_id
JOIN Departments d
```

```
ON c.department_id = d.department_id
ORDER BY s.student_id, c.course_id;
```

student_id	student_name	course_id	course_name	grade	department_name
101	Alice Smith	C101	Algorithms	A	Computer Science
101	Alice Smith	C102	Linear Algebra	B+	Mathematics
101	Alice Smith	C103	Quantum Mechanics	B+	Physics
101	Alice Smith	C104	Shakespearean Drama	A-	Literature
102	Bob Johnson	C102	Linear Algebra	B	Mathematics
102	Bob Johnson	C103	Quantum Mechanics	A	Physics
103	Clara Evans	C103	Quantum Mechanics	B-	Physics
103	Clara Evans	C104	Shakespearean Drama	A	Literature
104	David Kumar	C101	Algorithms	A+	Computer Science
104	David Kumar	C102	Linear Algebra	C+	Mathematics
104	David Kumar	C103	Quantum Mechanics	B	Physics
104	David Kumar	C104	Shakespearean Drama	B	Literature

Q3. EASY – Calculate the Average Assignment Score per Course

Requirement: Calculate the average assignment score per course and join suitable tables to get course name and average score.

A. DATABASE + ALL TABLE CREATION QUERIES

```
CREATE DATABASE learning_platform;
USE learning_platform;

CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(100),
    email VARCHAR(100),
    registration_date DATE
);

CREATE TABLE Instructor (
    instructor_id INT PRIMARY KEY,
    instructor_name VARCHAR(100),
    email VARCHAR(100),
    department VARCHAR(100)
);

CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100),
    instructor_id INT,
    start_date DATE,
    end_date DATE,
    FOREIGN KEY (instructor_id) REFERENCES Instructor(instructor_id)
);

CREATE TABLE Enrollment (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    enrollment_date DATE,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

CREATE TABLE Assignments (
    assignment_id INT PRIMARY KEY,
    course_id INT,
    assignment_title VARCHAR(150),
    due_date DATE,
    max_score INT,
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

CREATE TABLE Submissions (
    submission_id INT PRIMARY KEY,
    student_id INT,
    assignment_id INT,
    submission_date DATETIME,
    score DECIMAL(10,2),
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (assignment_id) REFERENCES Assignments(assignment_id)
);
```

B. ALL INSERT QUERIES – source test data

```
INSERT INTO Students VALUES
(1, 'Alice Johnson', 'alice@example.com', '2024-01-05'),
(2, 'Bob Smith', 'bob@example.com', '2023-12-01'),
(3, 'Charlie Brown', 'charlie@example.com', '2023-11-10'),
(4, 'David Williams', 'david@example.com', '2023-10-10'),
(5, 'Emma Davis', 'emma@example.com', '2024-02-01'),
(6, 'Frank Thomas', 'frank@example.com', '2024-03-05'),
(7, 'Grace Miller', 'grace@example.com', '2024-02-18');

INSERT INTO Instructor VALUES
(1, 'Dr. John Doe', 'john.doe@university.com', 'Computer Science'),
(2, 'Dr. Sarah Lee', 'sarah.lee@university.com', 'Mathematics'),
(3, 'Dr. Michael Johnson', 'michael.j@university.com', 'Physics');

INSERT INTO Courses VALUES
(1, 'Database Management', 1, '2024-01-10', '2024-06-30'),
(2, 'Machine Learning', 2, '2024-02-15', '2024-07-30'),
(3, 'Data Structures', 1, '2023-09-01', '2024-04-01'),
(4, 'Quantum Mechanics', 3, '2024-03-01', '2024-08-30'),
(5, 'Calculus II', 2, '2024-04-05', '2024-09-15');

INSERT INTO Enrollment VALUES
(1, 1, 1, '2024-01-12'),
(2, 3, 1, '2024-01-14'),
(3, 2, 2, '2024-02-20');
```

```
(4, 4, 2, '2024-02-22'),
(5, 5, 3, '2023-09-10'),
(6, 6, 3, '2023-10-05'),
(7, 7, 4, '2024-03-05'),
(8, 1, 4, '2024-04-10'),
(9, 2, 4, '2024-03-12'),
(10, 3, 5, '2024-04-15');

INSERT INTO Assignments VALUES
(1, 1, 'SQL Basics Assignment', '2024-02-10', 100),
(2, 1, 'Advanced SQL Queries', '2024-03-15', 100),
(3, 2, 'Linear Regression Project', '2024-03-01', 100),
(4, 2, 'Neural Networks Report', '2024-04-05', 100),
(5, 3, 'Sorting Algorithms', '2023-10-10', 100),
(6, 3, 'Graph Theory Assignment', '2023-12-15', 100),
(7, 4, 'Schrodinger’s Equation Analysis', '2024-04-01', 100),
(8, 5, 'Integration Techniques', '2024-05-10', 100);

INSERT INTO Submissions VALUES
(1, 1, 1, '2024-02-08 00:00:00', 85),
(2, 2, 1, '2024-02-09 00:00:00', 92),
(3, 3, 2, '2024-03-14 00:00:00', 78),
(4, 4, 2, '2024-03-15 00:00:00', 95),
(5, 5, 3, '2024-02-28 00:00:00', 89),
(6, 6, 4, '2024-04-03 00:00:00', 76),
(7, 1, 5, '2023-10-08 00:00:00', 80),
(8, 1, 6, '2023-12-14 00:00:00', 88),
(9, 2, 7, '2024-03-30 00:00:00', 94),
(10, 3, 8, '2024-05-08 00:00:00', 90);
```

C. ANSWER QUERY

```
SELECT
    c.course_name,
    AVG(s.score) AS avg_score
FROM Courses c
JOIN Assignments a
    ON c.course_id = a.course_id
JOIN Submissions s
    ON a.assignment_id = s.assignment_id
GROUP BY c.course_name
ORDER BY avg_score DESC;
```

course_name	avg_score
Quantum Mechanics	94.0000
Calculus II	90.0000
Database Management	87.5000
Data Structures	84.0000
Machine Learning	82.5000

Q4. MEDIUM – Highest-Revenue Doctor per Department

Requirement: For every doctor calculate department_name, doctor_name, total_revenue and revenue_rank. Use RANK(), not ROW_NUMBER(); equal revenue gets the same rank.

A. DATABASE + ALL TABLE CREATION QUERIES

```
CREATE DATABASE HOSPITAL_DB;
USE HOSPITAL_DB;

CREATE TABLE Departments (
    dept_id INT PRIMARY KEY,
    department_name VARCHAR(100)
);

CREATE TABLE Doctors (
    doctor_id INT PRIMARY KEY,
    doctor_name VARCHAR(100),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Departments(dept_id)
);

CREATE TABLE Appointments (
    appointment_id INT PRIMARY KEY,
    doctor_id INT,
    appointment_date DATE,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id)
);

CREATE TABLE Bills (
    bill_id INT PRIMARY KEY,
    appointment_id INT,
    amount DECIMAL(10,2),
    FOREIGN KEY (appointment_id) REFERENCES Appointments(appointment_id)
);
```

B. ALL INSERT QUERIES – source test data

```
INSERT INTO Departments VALUES
(1, 'Cardiology'),
(2, 'Neurology');

INSERT INTO Doctors VALUES
(1, 'Dr. Smith', 1),
(2, 'Dr. Anand', 1),
(3, 'Dr. Lee', 1),
(4, 'Dr. Patel', 2),
(5, 'Dr. Ray', 2);

INSERT INTO Appointments VALUES
(101, 1, '2025-11-15'),
(102, 1, '2025-11-15'),
(103, 2, '2025-11-16'),
(104, 2, '2025-11-16'),
(105, 3, '2025-11-14'),
(106, 4, '2025-11-17'),
(107, 4, '2025-11-17'),
(108, 5, '2025-11-17');

INSERT INTO Bills VALUES
(201, 101, 800.00),
(202, 102, 700.00),
(203, 103, 600.00),
(204, 104, 600.00),
(205, 105, 1200.00),
(206, 106, 1100.00),
(207, 107, 900.00),
(208, 108, 900.00);
```

C. ANSWER QUERY – WITHOUT SUBQUERY

```
SELECT
    d.department_name,
    dr.doctor_name,
    SUM(b.amount) AS total_revenue,
    RANK() OVER (
        PARTITION BY d.department_name
        ORDER BY SUM(b.amount) DESC
    ) AS revenue_rank
FROM Departments d
JOIN Doctors dr
    ON d.dept_id = dr.dept_id
JOIN Appointments a
    ON dr.doctor_id = a.doctor_id
JOIN Bills b
    ON a.appointment_id = b.appointment_id
GROUP BY
```

d.department_name,
dr.doctor_name;

department_name	doctor_name	total_revenue	revenue_rank
Cardiology	Dr. Smith	1500.00	1
Cardiology	Dr. Anand	1200.00	2
Cardiology	Dr. Lee	1200.00	2
Neurology	Dr. Patel	2000.00	1
Neurology	Dr. Ray	900.00	2

Q5. MEDIUM – Students Ranked by Course Enrollments

Requirement: Assign RANK() and DENSE_RANK() to each student based on number of courses, descending. Equal counts receive the same rank.
Final result is ordered by rank_by_enrollments.

A. DATABASE + ALL TABLE CREATION QUERIES

```
CREATE DATABASE university_core_rank;
USE university_core_rank;

CREATE TABLE Departments (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(100)
);

CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    age INT,
    department_id INT,
    FOREIGN KEY (department_id) REFERENCES Departments(department_id)
);

CREATE TABLE Courses (
    course_id VARCHAR(10) PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    department_id INT,
    FOREIGN KEY (department_id) REFERENCES Departments(department_id)
);

CREATE TABLE Enrollments (
    student_id INT,
    course_id VARCHAR(10),
    grade VARCHAR(5),
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

B. ALL INSERT QUERIES

```
INSERT INTO Departments VALUES
(1, 'Computer Science'),
(2, 'Mathematics'),
(3, 'Physics'),
(4, 'Literature');

INSERT INTO Students VALUES
(101, 'Alice Smith', 20, 1),
(102, 'Bob Johnson', 22, 2),
(103, 'Clara Evans', 21, 3),
(104, 'David Kumar', 23, 1);

INSERT INTO Courses VALUES
('C101', 'Algorithms', 1),
('C102', 'Linear Algebra', 2),
('C103', 'Quantum Mechanics', 3),
('C104', 'Shakespearean Drama', 4);

INSERT INTO Enrollments VALUES
(101, 'C101', 'A'),
(101, 'C102', 'B+'),
(101, 'C104', 'A-'),
(102, 'C102', 'B'),
(102, 'C103', 'A'),
(103, 'C103', 'B-'),
(103, 'C104', 'A'),
(104, 'C101', 'A+'),
(104, 'C103', 'B'),
(104, 'C102', 'C+'),
(104, 'C104', 'B'),
(101, 'C103', 'B+');
```

C. ANSWER QUERY – WITHOUT SUBQUERY

```
SELECT
    s.student_id,
    s.name,
    COUNT(e.course_id) AS course_count,
    RANK() OVER (
        ORDER BY COUNT(e.course_id) DESC
    ) AS rank_by_enrollments,
    DENSE_RANK() OVER (
        ORDER BY COUNT(e.course_id) DESC
    ) AS dense_rank_by_enrollments
FROM Students s
JOIN Enrollments e
```

```
        ON s.student_id = e.student_id
GROUP BY
    s.student_id,
    s.name
ORDER BY rank_by_enrollments;
```

student_id	name	course_count	rank_by_enrollments	dense_rank_by_enrollments
101	Alice Smith	4	1	1
104	David Kumar	4	1	1
102	Bob Johnson	2	3	2
103	Clara Evans	2	3	2

Q6. MEDIUM – Movies Watched by Viewers of 'Life of Pi'

Requirement: List movie_id and title for movies watched by any user who has also watched 'Life of Pi'. Duplicate movie titles must not appear.

A. DATABASE + ALL TABLE CREATION QUERIES

```
CREATE DATABASE movie_streaming;
USE movie_streaming;

CREATE TABLE Users (
    user_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    signup_date DATE NOT NULL
);

CREATE TABLE Genres (
    genre_id INT PRIMARY KEY,
    genre_name VARCHAR(50) NOT NULL
);

CREATE TABLE Movies (
    movie_id INT PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    release_year INT,
    genre_id INT,
    FOREIGN KEY (genre_id) REFERENCES Genres(genre_id)
);

CREATE TABLE Watch_History (
    user_id INT,
    movie_id INT,
    watch_date DATE,
    PRIMARY KEY (user_id, movie_id, watch_date),
    FOREIGN KEY (user_id) REFERENCES Users(user_id),
    FOREIGN KEY (movie_id) REFERENCES Movies(movie_id)
);

CREATE TABLE Subscriptions (
    subscription_id INT PRIMARY KEY,
    user_id INT,
    plan VARCHAR(50),
    amount DECIMAL(10,2),
    start_date DATE,
    end_date DATE NOT NULL,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
```

B. ALL INSERT QUERIES – source test data

```
INSERT INTO Users VALUES
(801, 'Alice Moore', 'alice@stream.com', '2023-01-15'),
(802, 'Bob Green', 'bob@stream.com', '2023-02-10'),
(803, 'Carol White', 'carol@stream.com', '2023-03-20'),
(804, 'Dave Black', 'dave@stream.com', '2023-04-05');

INSERT INTO Genres VALUES
(1, 'Drama'),
(2, 'Action'),
(3, 'Comedy'),
(4, 'Documentary');

INSERT INTO Movies VALUES
(901, 'The Great Escape', 1963, 2),
(902, 'Life of Pi', 2012, 4),
(903, 'The Office (Series)', 2005, 3),
(904, 'Hamlet', 1996, 1);

INSERT INTO Watch_History VALUES
(801, 901, '2023-08-01'),
(801, 902, '2023-08-02'),
(802, 903, '2023-08-03'),
(802, 904, '2023-08-04'),
(803, 901, '2023-08-05'),
(803, 902, '2023-08-06'),
(804, 903, '2023-08-07'),
(804, 904, '2023-08-08'),
(801, 904, '2023-08-09'),
(802, 902, '2023-08-10'),
(803, 904, '2023-08-11'),
(804, 901, '2023-08-12');

INSERT INTO Subscriptions VALUES
(1001, 801, 'Premium', 19.99, '2023-08-01', '2023-08-31'),
(1002, 802, 'Basic', 9.99, '2023-08-01', '2023-08-31'),
(1003, 803, 'Premium', 19.99, '2023-08-01', '2023-08-31'),
(1004, 804, 'Standard', 14.99, '2023-08-01', '2023-08-31');
```

C. ANSWER QUERY – WITHOUT SUBQUERY

```
SELECT DISTINCT
  m.movie_id,
  m.title
FROM Watch_History wh
JOIN Watch_History wh2
  ON wh.user_id = wh2.user_id
JOIN Movies m
  ON wh.movie_id = m.movie_id
JOIN Movies m2
  ON wh2.movie_id = m2.movie_id
WHERE m2.title = 'Life of Pi';
```

movie_id	title
901	The Great Escape
902	Life of Pi
903	The Office (Series)
904	Hamlet

IMPORTANT EXAM NOTE

This PDF now includes, for every question: database/table creation SQL, INSERT SQL for the test data, the answer query, and expected output where the source PDF provides it. Q4, Q5 and Q6 use the no-subquery versions requested in this conversation. Q1 keeps the source requirement of a single-row subquery. The source PDF contains 22 pages and six SQL questions. ■filecite■turn3file0■L1-L9■